Programmation PHP

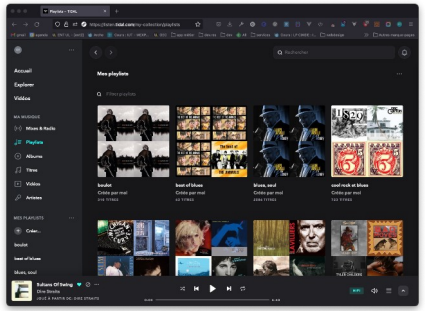
transmettre des données à l'application, formulaires

Principes

- Technique 1 : dans la partie query_string de l'url
  - `index.php?p1=toto&p2=titi`
  - utilisable avec n'importe quelle requête http (GET, POST, PUT, DELETE)

- Dans le programme php : accessible dans le tableau superglobal `$_GET,`
  - `$_GET[ 'p1'] , $_GET['p2']`
  - Vérifier l'existence d'un paramètre : `isset($_GET['p3'])`

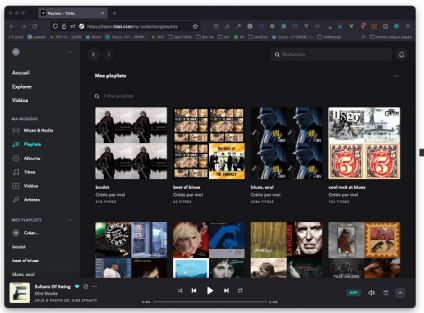
- Technique 2 : dans le corps d'une requête HTTP POST
- A partir d'un navigateur : forcément associée à un **formulaire html**
- Dans le programme PHP Les données sont disponibles dans le tableau $_POST
  - `$_POST[ 'p1']`



```
GET /add?nom=toto&age=73 HTTP/1.1
Host: www.crazynet.org
```

serveur

```
$nom=$_GET['nom'];
$age=$_GET['age'];
```



```
POST /add HTTP/1.1
Host: www.crazynet.org

nom=toto&age=73
```

serveur

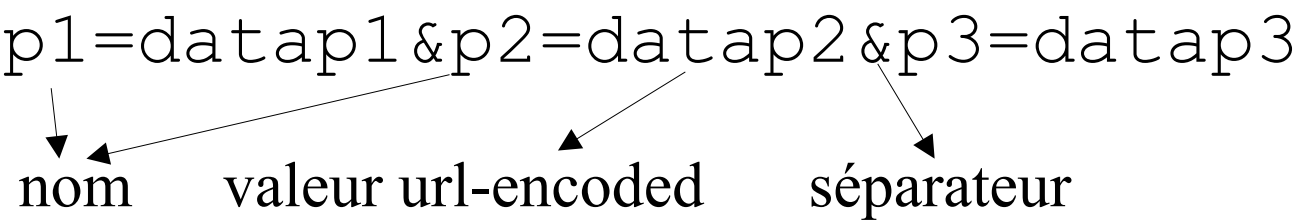
```
$nom=$_POST['nom'];
$age=$_POST['age'];
```

Principes : utiliser GET ou POST ?

- GET est une méthode HTTP "**safe**" : elle ne doit pas modifier l'état de l'application
  - **Utiliser GET pour des opérations de navigation/recherche dans les données de l'application**
  - **ne pas utiliser GET pour l'ajout, la suppression ou la modification de données**
- POST est une méthode HTTP "**unsafe**" : elle peut modifier l'état de l'application
  - **utiliser POST pour les actions conduisant à une modification, suppression, insertion de données**

Encodage des données

- Les données transmises dans la query string de l'url ou dans le body de la requête sont structurées encodées de la même manière : "url-encoded"



- Encodage des valeurs : string avec remplacement de certains caractères spéciaux par %xx :

`ab cd'=ef@?33;&` ⟶ `ab%20cd%27%3Def%40%3F33%3B%26`

- Type (mime) : `application/x-www-form-urlencoded`

saisir des données dans le navigateur : les formulaires html

- le formulaire :
- les contrôles :

```
<form id="form1"
      method="POST" action="/add">

<input type="text"
       name="nom"
       value="michel">
…
</form>
```



```
POST /add HTTP/1.1
Host: www.crazynet.org

nom=loulou
```

<input>

- Les contrôles interactifs simples : la balise input
  - `<input type... name...  />`
- Les attributs :
  - type= "…": spécifie le type de valeur attendue -
    - `text` : du texte simple
    - `password` : idem mais ******
    - `search` : presque idem
    - `number, url, email, date, color`
  - `name= ''…''`: nom du contrôle, entrée dans $_POST
  - `value= ''…''`: valeur initiale du contrôle, affichée dans le formulaire

les <inputs> : attributs optionnels

- `required` : saisie obligatoire de la valeur
- `placeholder` = "..." : un texte affiché dans le contrôle pour aider la saisie
- `autocomplete` : autorise l'autocomplétion par le navigateur
- `autofocus` : donne le focus au contrôle

Quelques types d'inputs

```
<form id="f1">


    <input type="text"  placeholder="<text>" >
    <input type="url"  placeholder="<url>" >
    <input type="email"  placeholder="<email>" >
    <input type="number"  placeholder="<number>" >
    <input type="tel"  placeholder="<tel>" >

    <input type="color"  placeholder="<tel>" >
    <input type="date"  placeholder="<date>" >


    <button type="submit">valider</button>

</form>
```

https://codepen.io/guenego/pen/bQyJOa

formulaire élémentaire

```
<form id="f1" method="get" action="adduser.php">

    <label for="f1_name"> nom: </label>
    <input type="text" id="f1_name" name="username"
    placeholder="<name>" required>


    <button type="submit" name="valider_inscription"
    value="valid_f1">valider</button>

</form>
```

- label : associé à 1 contrôle
- button : pour valider, déclenche l'envoi des données
- https://codepen.io/guenego/pen/bGMKNNm

fieldset / legend

- pour regrouper des contrôles
  - https://codepen.io/guenego/pen/KrLLoj

```
6    <fieldset>
7      <legend> input texte avancés</legend>
8      <input type="text"  placeholder="<text>" >
9      <input type="url"  placeholder="<url>" >
10     <input type="email"  placeholder="<email>" >
11     <input type="number"  placeholder="<number>" >
12     <input type="tel"  placeholder="<tel>" >
13   </fieldset>
14
15    <fieldset>
16      <legend> colorpicker / datepicker</legend>
17     <input type="color"  placeholder="<tel>" >
18     <input type="date"  placeholder="<date>" >
19    </fieldset>
20     <button type="submit">valider</button>
21
```

listes, boutons radio

- liste : <select> + <option>
- bouton radio : les regrouper en leur donnant le même *name*
- https://codepen.io/guenego/pen/BGeeOG

```
<select name="select">
  <option value="value1">Valeur 1</option>
  <option value="value2" selected>Valeur 2</option>
  <option value="value3">Valeur 3</option>
</select>

  <label>marcel</label><input type="radio" name="groupe-radio1" value=1>
  <label>jeanine</label><input type="radio" name="groupe-radio1" value=2>
  <label>susy</label><input type="radio" name="groupe-radio1" value=3>
  <label>auguste</label><input type="radio" name="groupe-radio1" value=4>
```

le problème de la sécurité

- Il est **INDISPENSABLE** de systématiquement se méfier des données transmises à l'application depuis l'extérieur : formulaires, urls etc …
- **les risques** : elles peuvent contenir des tentatives d'injection
- **Injection SQL** : données contenant du code SQL pour attaquer la BD => requêtes préparées
- **Injection HTML ou XSS** : données contenant du code HTML/JS exécuté au moment où les données sont affichées par l'application

Cross-Site scripting : XSS

- Principe : insérer des balises html avec des scripts dans des données qui sont affichées par l'application

```
<?
    echo "votre recherche {$_POST['query']} a échoué <br>" ;
```

- Dans le formulaire, on saisit :
  - **"<script> alert("Coucou Bilou");</script>**
  - Ou un script plus méchant !
- Ce script est exécuté par le navigateur

se protéger

Il faut FILTRER TOUTES les données en provenance de l'extérieur de l'application

en php : filter_var($var, $filter, $options)

- Appliquer avant chaque utilisation d'une donnée extraite de la requête ou de l'URL
- reçoit une variable, un id de filtre, un tableau d'options
- retourne la donnée filtrée ou false
- 2 types de filtres
  - filtres de validation: la fonction retourne false si la donnée n'est pas valide par rapport au filtre
  - filtres de nettoyage : la fonction modifie la donnée en appliquant le filtre

filtres de validation

- FILTER_VALIDATE_EMAIL
- FILTER_VALIDATE_URL
- FILTER_VALIDATE_INT
- FILTER_VALIDATE_IP
- FILTER_REGEXP
- ...

```
<?
 $email =$_POST[ 'email_user' ] ;
 if (! filter_var($email, FILTER_VALIDATE_EMAIL) ) {
    echo "adresse email invalide !";
    … // surtout ne pas afficher la variable !!
```

filtres de nettoyage

- filtres transformant la données en supprimant des caractères sensibles, notamment les balises, ou inappropriés
- `FILTER_SANITIZE_EMAIL`, `FILTER_SANITIZE_URL` :
  - suppriment tous les caractères sauf lettre, chiffres et caractères autorisés dans une @email/url
- `FILTER_SANITIZE_NUMBER_INT`, `FILTER_SANITIZE_NUMBER_FLOAT`
  - suppriment tous les caractères sauf les chiffres, +,-, ., E

- Nettoyage des chaînes de caractères : il faut en particulier traiter les éventuelles balises HTML

- `FILTER_SANITIZE_SPECIAL_CHARS`
  - transforme les caractères ",',<,>& pour les rendre affichables : « < » devient « < » etc ..

```
<?
  $query = filter_var($_POST['query'] ,
                      FILTER_SANITIZE_SPECIAL_CHARS) ;
  echo "votre recherche $query a échoué <br>" ;
```

transférer des fichiers : côté client

- input de type `file`
- encodage des données : `multipart/form-data`
- https://codepen.io/guenego/pen/xxjzbVE

```
<form id="form-add"  method="post"
    action="addpicture.php"
   enctype="multipart/form-data">

 <input type="file" name="inputfile">
 <br><button type="submit" name="valider"
   value="validation-form-add">transférer</button>
</form>
```

Réception de fichiers uploadés

- Les fichiers téléchargés par un client vers le serveur (*uploadés*) sont stockés par le serveur avec un nom temporaire. Des informations décrivant ce fichier sont disponibles dans le tableau superglobal `$_FILES`
- Lors de la réception d'un fichier :
  - Vérifier qu'il s'agit bien d'un fichier uploadé
  - Vérifier éventuellement quelques caractéristiques : taille, type
  - Générer un nom unique pour ce fichier
  - Déplacer le fichier vers un répertoire de stockage non accessible depuis une url (sauf besoin explicite, par exemple des images

- Le tableau $_FILES : `'inputfile'` = name de l'input HTML
  - `$FILES['inputfile']['tmp_name']`: nom temporaire (serveur)
  - `$FILES['inputfile']['name']` : nom original (client)
  - `$FILES['inputfile']['size']`: taille en octets
  - `$FILES['inputfile']['type']`: type mime e.g. "image/png"
  - `$FILES['inputfile']['error']`: code d'erreur (0 OK)
- Fonctions utiles :

```
is_uploaded_file(string $tmp_filename) : bool
```

S'assure que le fichier a été transféré avec une requête POST

```
move_uploaded_file(string $tp, string $dest):bool
```

S'assure que le fichier a été transféré avec une requête POST. Si c'est le cas, le fichier est déplacé vers $dest

```php
$upload_dir = __DIR__ . '/img/';
$filename = uniqid();

$tmp = $_FILES['inputfile']['tmp_name'] ;

if (($_FILES['inputfile']['error'] === UPLOAD_ERR_OK) &&
    ($_FILES['inputfile']['type'] === 'image/png') ) {

$dest = $upload_dir.$filename.'.png';

      if (move_uploaded_file($tmp, $dest )) {

          print "téléchargement terminé avec succès<br>";
      } else {
          print "hum, hum téléchargement non valide<br>";
      }

    } else {
    print "echec du téléchargement ou type non autorisé<br>";
}
```